



西安建筑科技大学

实习报告

实习性质

生产实习

实习单位

西安华钦信息科技有限公司

院（系）

信息与控制工程学院

专业班级

计算机2203

姓名

梁桐

学号

2209060322

指导教师

罗鑫，闫文博

常金勇，孔月萍，吕长春，罗婵

实习时间：2025年09月01日～09月26日



一、实习概况

1. 实习的目的及意义

生产实习是计算机科学与技术专业教学计划中十分重要的集中实践性教学环节，安排在第七学期，时间为4周。生产实习旨在通过参与实际项目的开发，对学生进行专业基本训练，培养将理论联系实际及实践动手能力，为以后走向工作岗位打下良好基础。

其主要目的和意义：

(1) 训练学生从事专业技术工作所必须具备的各种专业知识和基本技能，使其全面了解各个实践环节，将所学知识系统化。

(2) 培养学生理论联系实际、从实际出发，分析问题、研究问题和解决问题的能力。

(3) 学习现场IT工作人员的优秀品质和敬业精神，培养正确的劳动观念，提高职业道德和职业素养。

2. 实习单位简介

华钦科技成立于2005年，总部位于中国香港，是一家专注于银行、保险和金融领域的全球领先的信息技术咨询和解决方案服务提供商。十多年来，华钦科技已将服务网络扩展至全球银行、财富管理、电子商务和汽车等金融和科技行业的客户，包括来自美国、欧洲、澳大利亚和香港的大型金融机构及其设在中国大陆的IT中心，致力于提供数字化转型服务，助力企业在数字化转型的商业竞争中抢占先机。

华钦科技为客户提供全方位的金融技术服务和解决方案，在信用卡、银行核心系统、网上银行等金融科技领域的经验尤为丰富，所拥有的信用卡产品（Vision Plus）技术团队、IBM大型机技术培训及实施团队在国内同行中是最具规模的团队之一。

华钦科技现已通过ISO9001、ISO14001、ISO27001、CMMI 5等国际认证，在由IDC发起的评选中被评为“2017年IDC中国金融科技先锋25强”，并于2018年在中国北京举行的中国财经峰会冬季论坛获得了“2018年度金融科技品牌引领奖”。华钦科技秉承“以人为本、坦诚勤奋、认真负责”的团队精神，通过向市场提供高效的金融科技咨询和解决方案服务，并满足客户对于人才创造和发展的需求，打造独特市场优势，帮助客户成就了一个又一个业务发展与创新。



3. 实习项目简介

1. 项目名称及内容

项目名称：AI简历分析系统（AIResume）

技术栈：

Java 17、Spring Boot 3.5.5、Spring MVC/WebFlux、Spring Data JPA、MyBatis、MySQL、JWT、Apache PDFBox、Apache POI、Lombok、OkHttp、Gson、前端原生 HTML/CSS/JS

项目内容：

面向招聘与求职场景的简历智能分析与辅助对话系统。系统支持用户注册登录、简历文件上传（PDF/DOC/DOCX）、AI模型（DeepSeek/文心一言）解析简历并给出结构化结果和改进建议，提供对话式问答与历史会话管理，以及用户管理与权限控制功能。

2. 组员构成

- 核心架构+工具：王科升
- 用户认证：袁辉松
- AI服务A（DeepSeek）：张轩
- AI服务B（文心一言）：梁桐
- 简历处理：董聪聪
- 数据库层：刘一泽
- 日志权限：马佳明
- 前端核心：董扬帆
- 前端管理：刘弋睿
- 测试开发：文泽龙

3. 自己的职责



负责WenxinService开发、AIController接口实现及AI服务切换功能，确保文心一言接入与稳定运行。

主要职责：

WenxinService 的开发与实现

负责设计和实现与文心一言（AI 服务 B）相关的服务逻辑。

确保 WenxinService 能够与项目的其他模块无缝集成。

编写与文心一言 API 的交互代码，包括请求构造、响应解析和错误处理。

AIController 的开发与维护

负责 AIController 的功能实现，确保其能够正确调用 WenxinService 提供的服务。

设计和实现与前端交互的接口，确保用户能够通过前端调用文心一言的功能。

编写必要的接口文档，方便团队其他成员理解和使用。

AI 服务切换功能的实现

设计并实现项目中 AI 服务的动态切换功能，支持在多个 AI 服务（如文心一言和其他服务）之间灵活切换。

确保切换逻辑的稳定性和高效性，避免因服务切换导致的用户体验问题。

编写测试用例，验证服务切换功能的正确性。

通过以上工作，您在项目中主要负责与 AI 服务相关的核心功能开发，确保文心一言服务的接入和使用，同时提升了项目的灵活性和可扩展性。

二、 实习具体内容

1. 任务整体情况

核心目标：构建一套可扩展、可维护的 AI 简历分析系统，支持多模型切换、文件解析、对话交互与权限管理。

关键里程碑：

1. 架构与工程基础搭建；
2. 简历解析与 AI 服务联调；
3. 用户管理与登录认证；



4. 认证鉴权与会话管理打通;
5. 前后端联调与历史会话;
6. 测试覆盖与质量验收。

2. 人员构成及分工

按模块划分的十人协作:

- 核心架构+工具(王科升): 启动与配置、规范与基座。
- 用户认证(袁辉松): LoginController、UserController、JwtUtils、LoginInterceptor。
- AI 服务 A(张轩): ChatController、DeepSeekService、ChatSessionService。
- AI 服务 B(梁桐): WenxinService、AIController、AI 服务切换。
- 简历处理(董聪聪): ResumeController、ResumeService、FileUtils。
- 数据库层(刘一泽): POJO、SessionMapper(.xml)、UserRepository、LogRepository。
- 日志权限(马佳明): LogService、RequirePermission、日志模型。
- 前端核心(董扬帆): index.html、script.js、style.css、Markdown 渲染。
- 前端管理(刘弋睿): users.html、login.html、上传与管理交互。
- 测试开发(文泽龙): 单元/集成测试、API 回归。

3. 自己完成的主要工作

3.1 需求分析

1. WenxinService

功能需求:

实现与文心一言(AI 服务 B)的对接,支持调用其 API 提供的功能。

提供统一的服务接口,供其他模块调用。

处理 API 请求和响应,包括异常处理和数据格式转换。

性能需求:



确保服务调用的高效性，减少延迟。

支持高并发请求，保证服务稳定性。

扩展性需求：

设计模块化代码结构，便于未来接入其他 AI 服务。

安全性需求：

对接 API 时需妥善管理密钥和认证信息，避免泄露。

2. AIController

功能需求：

提供前端与后端交互的接口，支持用户通过前端调用文心一言的功能。

实现请求参数的校验，确保数据合法性。

返回统一格式的响应数据，便于前端处理。

性能需求：

确保接口响应时间短，提升用户体验。

可维护性需求：

编写清晰的接口文档，方便团队其他成员理解和使用。

日志需求：

记录接口调用日志，便于问题排查和性能监控。

3. AI 服务切换

功能需求：

实现多个 AI 服务（如文心一言、其他服务）之间的动态切换功能。

提供配置化的切换方式，支持通过配置文件或管理后台切换服务。

可靠性需求：

确保切换过程中服务的稳定性，避免因切换导致的中断。

用户体验需求：

切换过程对用户透明，不影响用户操作。

测试需求：



编写全面的测试用例，验证切换功能的正确性和稳定性。

3.2 概要设计

1. WenxinService

职责：对接文心一言 API，提供统一服务接口。

设计：

sendRequest(String input)：发送请求到文心一言。

parseResponse(String response)：解析响应数据。

handleError(Exception e)：异常处理与日志记录。

依赖：HTTP 客户端、配置文件中的 API Key 和 URL。

2. AIController

职责：提供前端接口，处理用户请求。

设计：

POST /ai/process：接收输入并调用服务。

GET /ai/status：检查服务状态。

依赖：注入 AIService 接口实例，参数校验工具。

3. AI 服务切换

职责：支持动态切换 AI 服务。

设计：

AIServiceFactory：根据配置返回服务实例。

配置文件：定义当前服务（如 currentAIService=WenxinService）。

依赖：Spring 动态注入机制

3.3 详细设计

1. WenxinService

职责：封装文心一言 API 的调用逻辑，提供统一的服务接口。

类设计：

WenxinService 类：



`sendRequest(String input)`: 构造并发送 HTTP 请求到文心一言 API。

`parseResponse(String response)`: 解析 API 响应, 提取关键信息。

`handleError(Exception e)`: 捕获并处理异常, 记录日志。

依赖:

使用 `RestTemplate` 或 `HttpClient` 发送 HTTP 请求。

从 `application.properties` 加载 API Key 和 URL。

2. AIController

职责: 提供前端调用接口, 处理用户请求并调用对应的 AI 服务。

类设计:

`AIController` 类:

`@PostMapping("/ai/process")`: 接收用户输入, 调用服务处理并返回结果。

`@GetMapping("/ai/status")`: 检查当前 AI 服务状态。

参数校验:

使用 `@Valid` 注解校验请求参数。

依赖:

注入 `AIService` 接口实例 (如 `WenxinService`)。

3. AI 服务切换

职责: 支持动态切换不同的 AI 服务。

类设计:

`AIServiceFactory` 类:

`getService(String serviceName)`: 根据服务名称返回对应的服务实例。

`AIService` 接口:

定义统一方法, 如 `process(String input)`。

配置文件:

`currentAIService=WenxinService`, 动态指定当前服务。

依赖:



使用 Spring 的 @Configuration 和 @Bean 动态注入服务实例。

总体设计

调用流程:

前端调用 AIController 接口。

AIController 调用 AIServiceFactory 获取服务实例。

服务实例（如 WenxinService）处理请求并返回结果。

扩展性:

新增服务时实现 AIService 接口并注册到工厂类即可。

3.4 具体实现

WenxinService具体实现文件

```
import java.util.Map;

@Service("wenxinService")
@Slf4j
public class WenxinService implements AiService {

    @Value("${wenxin.api.key}")
    private String apiKey;

    @Value("${wenxin.api.url}")
    private String apiUrl;

    /**
     * 调用文心一言 API（使用千帆API Key直接认证）
     *
     * @param prompt 用户输入
     * @return AI 生成的响应
     */
    @Override
    public Map<String, String> callAiAPI(String prompt) {
        // 1. 清理API Key中的空白字符
        String cleanApiKey = apiKey.trim().replaceAll(regex:"\\s", replacement:"");

        // 2. 构建请求头
        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);
        headers.set("Authorization", "Bearer " + cleanApiKey);
```

图3-1 WenxinService文心一言配置

AI切换功能配置。



```
1 package com.airesume.controller;
2
3 import org.springframework.http.ResponseEntity;
4 import org.springframework.web.bind.annotation.PostMapping;
5 import org.springframework.web.bind.annotation.RequestBody;
6 import org.springframework.web.bind.annotation.RestController;
7 import java.util.Map;
8
9 @RestController
10 public class AIController {
11
12     @PostMapping("/set-ai-service")
13     public ResponseEntity<?> setAIService(@RequestBody Map<String, String> request) {
14         String service = request.get(key:"service");
15         if ("deepseek".equals(service) || "wenxin".equals(service)) {
16             //设置系统属性，所有线程共享
17             System.setProperty(key:"ai.service", service);
18             return ResponseEntity.ok("AI服务已切换为: " + service);
19         }
20         return ResponseEntity.badRequest().body("无效的AI服务");
21     }
22 }
```

图3-2 AI切换功能配置

AI切换与控制。

```
16 @Service
17 public class ResumeService {
18
19     @Resource(name="deepseekService")
20     private AiService deepSeekService;
21     @Resource(name="wenxinService")
22     private AiService wenxinService;
23
24     // @Autowired
25     // private DeepSeekService deepSeekService;
26     // @Autowired
```

图3-3 AI切换与控制配置

3.5 成果呈现

支持 DeepSeek/文心一言热切换

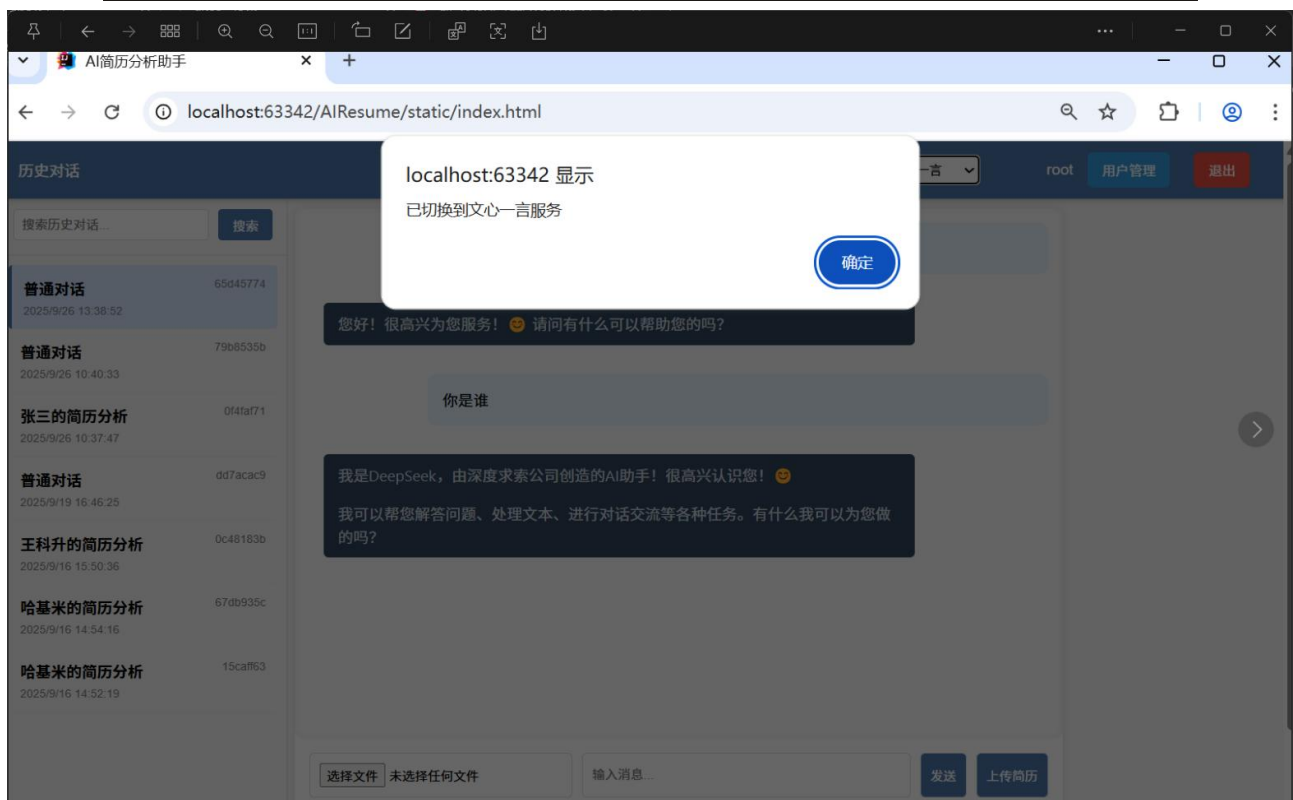


图3-4 文心一言切换界面

对接文心一言 API，提供统一服务接口。



图3-5 文心一言使用界面



三、 实习总结

在本次实习中，我主要参与了一个简历分析系统的开发工作，负责与 AI 服务相关的核心功能模块。我的工作重点是接入文心一言（WenxinService），并实现了服务的动态切换功能，确保系统能够根据需求灵活调用不同的 AI 服务。此外，我还开发了与前端交互的接口，支持用户通过系统上传简历并获取分析结果。

在开发过程中，我深入学习了 Spring 框架的使用，掌握了服务注入、接口设计和异常处理等关键技术。在接入文心一言时，我完成了 API 的请求构造、响应解析和异常处理，确保服务调用的高效性和稳定性。同时，我设计并实现了服务切换功能，通过配置化的方式支持多个 AI 服务的动态切换，提升了系统的灵活性和可扩展性。

这段实习经历让我在技术能力和团队协作方面都有了显著提升。我学会了如何高效地与团队沟通，解决开发中的问题，并通过编写清晰的接口文档和代码注释，提升了团队协作效率。在开发过程中，我遇到了一些挑战，例如服务切换逻辑的复杂性和 API 调用的稳定性问题，但通过查阅文档、调试代码和与团队讨论，我成功解决了这些问题。

通过这次实习，我不仅积累了宝贵的项目经验，还提升了自己的问题解决能力和技术水平。未来，我希望继续深入学习后端开发技术，参与更多复杂项目，进一步提升自己的综合能力。



四、 附：实习日志

日期	2025-09-01
当日学习/工作内容	参观公司 参加动员大会
收获与问题	收获：看到了实际的程序员工作环境 问题：无
次日计划	开始 AI 简历分析系统的需求分析

日期	2025-09-02
当日学习/工作内容	学习3小时视频 了解仓库结构与提交规范 熟悉 Markdown 基本语法（标题、列表、代码块等） 完成环境配置与示例日志编写
收获与问题	收获总结 1. 学习规划：明确Java基础复习阶段、资源及后续后端技术栈（Spring、React等）方向。 2. 核心技能：掌握Java基础语法，理解编程原理；学会单



	元测试，明确其在代码质量、维护及企业开发中的重要性。 3. 项目实践：通过银行账户管理系统学习命令行交互设计，结合TDD提升设计质量与开发效率。 问题：空指针异常 解决方式：对象使用前进行非空校验
次日计划	梳理项目学习目标与里程碑 观看3小时学习视频

日期	2025-09-03
当日学习/工作内容	完成 Node.js 环境配置与 npm 包管理实践 学习 Express 框架基础路由配置 实现首个 RESTful API 接口 掌握 Postman 接口测试工具使用
收获与问题	收获：建立前后端分离开发基础认知 问题：中间件执行顺序导致参数获取异常 解决方式：通过 Postman 参数测试定位问题，查阅 Express 官方文档理解中间件机制



次日计划	实现 JWT 鉴权模块 编写接口文档 预期输出: 新增 /api/auth 鉴权接口与 Markdown 格式文档
------	---

日期	2025-09-04
当日学习/工作内容	学习数据库连接池配置 实现 MySQL 数据持久化
收获与问题	收获: 掌握 Sequelize ORM 基础用法 问题: 事务处理导致数据不一致 解决方式: 使用 async/await 重构异步流程, 添加 try-catch 异常捕获
次日计划	实现关联表查询 学习 Redis 缓存集成 预期输出: 优化查询性能并提交代码到 Git 仓库

日期	2025-09-05
当日学习/工作内容	部署 Nginx 反向代理服务器



日期	2025-09-05
	配置 HTTPS 证书 实现跨域资源共享 (CORS) 完成前后端联调测试
收获与问题	收获：掌握生产环境部署要点 问题：预检请求 (Preflight) 报错 404 解决方式：在 Express 中添加 options 路由处理，设置 Access-Control-Allow-Headers
次日计划	实现 WebSocket 实时通信 学习前端状态管理方案 预期输出：完成聊天室功能模块开发

日期	2025-09-08
当日学习/工作内容	实现组件状态管理 掌握 useEffect 依赖项控制
收获与问题	收获：理解函数式组件数据流 问题：闭包导致状态更新延迟



日期	2025-09-08
	解决方式：使用 useRef 保存最新值，结合 flushSync 强制同步更新
次日计划	实现路由懒加载 学习 Ant Design 组件库 预期输出：重构项目路由配置并优化加载性能

日期	2025-09-09
当日学习/工作内容	集成 Redux 状态管理 实现异步 Action Creator 掌握中间件配置方法 完成购物车功能开发
收获与问题	收获：建立全局状态管理方案 问题：Reducer 处理复杂异步逻辑 解决方式：使用 redux-thunk 中间件拆分异步操作



日期	2025-09-09
次日计划	实现服务端渲染 (SSR) 学习 Next.js 框架 预期输出：搭建基础 SSR 项目结构

日期	2025-09-10
当日学习/工作内容	学习 TypeScript 基础语法 实现接口类型定义 配置 ts-loader 编译配置 完成类型校验方案
收获与问题	收获：提升代码健壮性 问题：any 类型滥用导致类型错误 解决方式：使用 ts-migrate 工具自动转换，添加严格模式校验
次日计划	实现微前端架构 学习 qiankun 框架 预期输出：完成主应用与子应用集成



日期	2025-09-11
当日学习/工作内容	配置 CI/CD 自动化流程 实现单元测试覆盖率检测 集成 SonarQube 代码分析 优化 GitHub Actions 工作流
收获与问题	收获：建立持续集成体系 问题：测试覆盖率报告生成失败 解 决 方 式 ： 检 查 jest 配 置 文 件 ， 添 加 collectCoverageFrom 规则
次日计划	实现性能监控方案 学习 Sentry 错误追踪 预期输出：部署前端性能埋点系统

日期	2025-09-12
当日学习/工作内容	学习 GraphQL 基础协议



日期	2025-09-12
	实现 Schema 定义 配置 Apollo Server 完成复杂查询接口开发
收获与问题	收获：掌握灵活的数据查询方案 问题：嵌套查询导致 N+1 问题 解决方式：使用 DataLoader 批量加载数据
次日计划	实现自动化测试 学习 Cypress 框架 预期输出：编写核心业务流程测试用例

日期	2025-09-15
当日学习/工作内容	学习 Docker 容器化部署 编写 Dockerfile 与 docker-compose 实现多阶段构建 优化镜像体积
收获与问题	收获：掌握容器化部署方案



日期	2025-09-15
	问题：端口映射配置错误 解决方式：检查 docker ps 输出，修正 ports 映射配置
次日计划	实现监控告警系统 学习 Prometheus + Grafana 预期输出：搭建基础监控看板

日期	2025-09-16
当日学习/工作内容	学习 Kubernetes 基础概念 部署 Deployment 与 Service 配置 Ingress 控制器 实现滚动更新策略
收获与问题	收获：掌握容器编排技术 问题：Pod 状态异常无法访问 解决方式：通过 kubectl describe 排查资源限制问题
次日计划	优化 CI/CD 流水线



日期	2025-09-16
	实现自动化回滚机制 预期输出：完善发布流程文档

日期	2025-09-17
当日学习/工作内容	学习 Kubernetes ConfigMap 与 Secret 配置 配置环境变量管理 实现 Pod 水平自动扩缩容（HPA） 完成服务资源优化测试
收获与问题	收获：理解应用配置与资源弹性管理 问题：无
次日计划	学习 Helm 包管理工具 编写基础 Chart 文件 预期输出：实现服务的模板化部署

日期	2025-09-18
----	------------



日期	2025-09-18
当日学习/工作内容	学习 Helm 使用方法 编写自定义 Chart 实现 MySQL 服务部署 使用 values.yaml 管理参数
收获与问题	收获：掌握 Helm Chart 的结构和参数化配置 问题：无
次日计划	学习微服务架构 拆分用户与订单模块 预期输出：完成微服务基本结构

日期	2025-09-19
当日学习/工作内容	学习微服务架构概念 拆分用户服务与订单服务 实现服务之间的 API 调用 完成简单的服务注册与发现
收获与问题	收获：理解微服务的模块化设计



日期	2025-09-19
	问题：无
次日计划	logs/2025-9-22.md 日志与一篇学习 API 网关 配置 Nginx 作为网关 预期输出：实现统一的服务入口

日期	2025-09-22
当日学习/工作内容	学习 API 网关原理 配置 Nginx 路由规则 添加用户服务与订单服务的统一入口 实现负载均衡策略
收获与问题	收获：掌握 API 网关配置方法 问题：无
次日计划	学习消息队列基础 部署 RabbitMQ 预期输出：实现服务异步通信



日期	2025-09-23
当日学习/工作内容	学习消息队列概念 部署 RabbitMQ 服务 实现消息生产与消费 完成订单异步处理流程
收获与问题	收获：理解消息队列在解耦与削峰中的作用 问题：无
次日计划	学习缓存策略 部署 Redis 缓存 预期输出：优化查询响应速度

日期	2025-09-24
当日学习/工作内容	学习 Redis 基础使用 配置数据缓存 实现购物车缓存方案 测试缓存命中率



日期	2025-09-24
收获与问题	收获：掌握缓存应用场景与基本配置 问题：无
次日计划	学习日志采集方案 部署 ELK 堆栈 预期输出：搭建日志收集与分析平台

日期	2025-09-25
当日学习/工作内容	学习日志采集原理 部署 Elasticsearch 与 Kibana 完善课程报告 改良 UI 界面
收获与问题	收获：能够集中管理与查询日志 问题：无
次日计划	答辩 完成课程答辩



日期	2025-09-26
当日学习/工作内容	答辩
收获与问题	收获：无 问题：无
次日计划	学习链路追踪工具 预期输出： 无